

Chapter 12. Optimization

Optimization is the process of making your application run more effectively. You can optimize for many things - speed, memory space usage, disk space usage, etc. This chapter, however, focuses on speed optimization.

When to Optimize

It is better to not optimize at all than to optimize too soon. When you optimize, your code generally becomes less clear, because it becomes more complex. Readers of your code will have more trouble discovering why you did what you did which will increase the cost of maintenance of your project. Even when you know how and why your program runs the way it does, optimized code is harder to debug and extend. It slows the development process down considerably, both because of the time it takes to optimize the code, and the time it takes to modify your optimized code.

Compounding this problem is that you don't even know beforehand where the speed issues in your program will be. Even experienced programmers have trouble predicting which parts of the program will be the bottlenecks which need optimization, so you will probably end up wasting your time optimizing the wrong parts. the Section called *Where to Optimize* will discuss how to find the parts of your program that need optimization.

While you develop your program, you need to have the following priorities:

- Everything is documented
- Everything works as documented
- The code is written in an modular, easily modifiable form

Documentation is essential, especially when working in groups. The proper functioning of the program is essential. You'll notice application speed was not

Chapter 12. Optimization

anywhere on that list. Optimization is not necessary during early development for the following reasons:

- Minor speed problems can be usually solved through hardware, which is often much cheaper than a programmer's time.
- Your application will change dramatically as you revise it, therefore wasting most of your efforts to optimize it.¹
- Speed problems are usually localized in a few places in your code - finding these is difficult before you have most of the program finished.

Therefore, the time to optimize is toward the end of development, when you have determined that your correct code actually has performance problems.

In a web-based e-commerce project I was involved in, I focused entirely on correctness. This was much to the dismay of my colleagues, who were worried about the fact that each page took twelve seconds to process before it ever started loading (most web pages process in under a second). However, I was determined to make it the right way first, and put optimization as a last priority. When the code was finally correct after 3 months of work, it took only three days to find and eliminate the bottlenecks, bringing the average processing time under a quarter of a second. By focusing on the correct order, I was able to finish a project that was both correct and efficient.

Where to Optimize

Once you have determined that you have a performance issue you need to determine where in the code the problems occur. You can do this by running a *profiler*. A profiler is a program that will let you run your program, and it will tell you how much time is spent in each function, and how many times they are run.

1. Many new projects often have a first code base which is completely rewritten as developers learn more about the problem they are trying to solve. Any optimization done on the first codebase is completely wasted.

gprof is the standard GNU/Linux profiling tool, but a discussion of using profilers is outside the scope of this text. After running a profiler, you can determine which functions are called the most or have the most time spent in them. These are the ones you should focus your optimization efforts on.

If a program only spends 1% of its time in a given function, then no matter how much you speed it up you will only achieve a *maximum* of a 1% overall speed improvement. However, if a program spends 20% of its time in a given function, then even minor improvements to that functions speed will be noticeable. Therefore, profiling gives you the information you need to make good choices about where to spend your programming time.

In order to optimize functions, you need to understand in what ways they are being called and used. The more you know about how and when a function is called, the better position you will be in to optimize it appropriately.

There are two main categories of optimization - local optimizations and global optimizations. Local optimizations consist of optimizations that are either hardware specific - such as the fastest way to perform a given computation - or program-specific - such as making a specific piece of code perform the best for the most often-occurring case. Global optimization consist of optimizations which are structural. For example, if you were trying to find the best way for three people in different cities to meet in St. Louis, a local optimization would be finding a better road to get there, while a global optimization would be to decide to teleconference instead of meeting in person. Global optimization often involves restructuring code to avoid performance problems, rather than trying to find the best way through them.

Local Optimizations

The following are some well-known methods of optimizing pieces of code. When using high level languages, some of these may be done automatically by your compiler's optimizer.

Chapter 12. Optimization

Precomputing Calculations

Sometimes a function has a limited number of possible inputs and outputs. In fact, it may be so few that you can actually precompute all of the possible answers beforehand, and simply look up the answer when the function is called. This takes up some space since you have to store all of the answers, but for small sets of data this works out really well, especially if the computation normally takes a long time.

Remembering Calculation Results

This is similar to the previous method, but instead of computing results beforehand, the result of each calculation requested is stored. This way when the function starts, if the result has been computed before it will simply return the previous answer, otherwise it will do the full computation and store the result for later lookup. This has the advantage of requiring less storage space because you aren't precomputing all results. This is sometimes termed *caching* or *memoizing*.

Locality of Reference

Locality of reference is a term for where in memory the data items you are accessing are. With virtual memory, you may access pages of memory which are stored on disk. In such a case, the operating system has to load that memory page from disk, and unload others to disk. Let's say, for instance, that the operating system will allow you to have 20k of memory in physical memory and forces the rest of it to be on disk, and your application uses 60k of memory. Let's say your program has to do 5 operations on each piece of data. If it does one operation on every piece of data, and then goes through and does the next operation on each piece of data, eventually every page of data will be loaded and unloaded from the disk 5 times. Instead, if you did all 5 operations on a given data item, you only have to load each page from disk once. When you bundle as many operations on data that is physically close to each other in memory, then you are taking advantage of locality of reference.

In addition, processors usually store some data on-chip in a cache. If you keep all of your operations within a small area of physical memory, your program may bypass even main memory and only use the chip's ultra-fast cache memory. This is all done for you - all you have to do is to try to operate on small sections of memory at a time, rather than bouncing all over the place.

Register Usage

Registers are the fastest memory locations on the computer. When you access memory, the processor has to wait while it is loaded from the memory bus. However, registers are located on the processor itself, so access is extremely fast. Therefore making wise usage of registers is extremely important. If you have few enough data items you are working with, try to store them all in registers. In high level languages, you do not always have this option - the compiler decides what goes in registers and what doesn't.

Inline Functions

Functions are great from the point of view of program management - they make it easy to break up your program into independent, understandable, and reuseable parts. However, function calls do involve the overhead of pushing arguments onto the stack and doing the jumps (remember locality of reference - your code may be swapped out on disk instead of in memory). For high level languages, it's often impossible for compilers to do optimizations across function-call boundaries. However, some languages support inline functions or function macros. These functions look, smell, taste, and act like real functions, except the compiler has the option to simply plug the code in exactly where it was called. This makes the program faster, but it also increases the size of the code. There are also many functions, like recursive functions, which cannot be inlined because they call themselves either directly or indirectly.

Chapter 12. Optimization

Optimized Instructions

Often times there are multiple assembly language instructions which accomplish the same purpose. A skilled assembly language programmer knows which instructions are the fastest. However, this can change from processor to processor. For more information on this topic, you need to see the user's manual that is provided for the specific chip you are using. As an example, let's look at the process of loading the number 0 into a register. On most processors, doing a `movl $0, %eax` is not the quickest way. The quickest way is to exclusive-or the register with itself, `xorl %eax, %eax`. This is because it only has to access the register, and doesn't have to transfer any data. For users of high-level languages, the compiler handles this kind of optimizations for you. For assembly-language programmers, you need to know your processor well.

Addressing Modes

Different addressing modes work at different speeds. The fastest are the immediate and register addressing modes. Direct is the next fastest, indirect is next, and base pointer and indexed indirect are the slowest. Try to use the faster addressing modes, when possible. One interesting consequence of this is that when you have a structured piece of memory that you are accessing using base pointer addressing, the first element can be accessed the quickest. Since it's offset is 0, you can access it using indirect addressing instead of base pointer addressing, which makes it faster.

Data Alignment

Some processors can access data on word-aligned memory boundaries (i.e. - addresses divisible by the word size) faster than non-aligned data. So, when setting up structures in memory, it is best to keep it word-aligned. Some non-x86 processors, in fact, cannot access non-aligned data in some modes.

These are just a smattering of examples of the kinds of local optimizations possible. However, remember that the maintainability and readability of code is

much more important except under extreme circumstances.

Global Optimization

Global optimization has two goals. The first one is to put your code in a form where it is easy to do local optimizations. For example, if you have a large procedure that performs several slow, complex calculations, you might see if you can break parts of that procedure into their own functions where the values can be precomputed or memoized.

Stateless functions (functions that only operate on the parameters that were passed to them - i.e. no globals or system calls) are the easiest type of functions to optimize in a computer. The more stateless parts of your program you have, the more opportunities you have to optimize. In the e-commerce situation I wrote about above, the computer had to find all of the associated parts for specific inventory items. This required about 12 database calls, and in the worst case took about 20 seconds. However, the goal of this program was to be interactive, and a long wait would destroy that goal. However, I knew that these inventory configurations do not change. Therefore, I converted the database calls into their own functions, which were stateless. I was then able to memoize the functions. At the beginning of each day, the function results were cleared in case anyone had changed them, and several inventory items were automatically preloaded. From then on during the day, the first time someone accessed an inventory item, it would take the 20 seconds it did beforehand, but afterwards it would take less than a second, because the database results had been memoized.

Global optimization usually often involves achieving the following properties in your functions:

Parallelization

Parallelization means that your algorithm can effectively be split among multiple processes. For example, pregnancy is not very parallelizable because

Chapter 12. Optimization

no matter how many women you have, it still takes nine months. However, building a car is parallelizable because you can have one worker working on the engine while another one is working on the interior. Usually, applications have a limit to how parallelizable they are. The more parallelizable your application is, the better it can take advantage of multiprocessor and clustered computer configurations.

Statelessness

As we've discussed, stateless functions and programs are those that rely entirely on the data explicitly passed to them for functioning. Most processes are not entirely stateless, but they can be within limits. In my e-commerce example, the function wasn't entirely stateless, but it was within the confines of a single day. Therefore, I optimized it as if it were a stateless function, but made allowances for changes at night. Two great benefits resulting from statelessness is that most stateless functions are parallelizable and often benefit from memoization.

Global optimization takes quite a bit of practice to know what works and what doesn't. Deciding how to tackle optimization problems in code involves looking at all the issues, and knowing that fixing some issues may cause others.

Review

Know the Concepts

- At what level of importance is optimization compared to the other priorities in programming?
- What is the difference between local and global optimizations?
- Name some types of local optimizations.

- How do you determine what parts of your program need optimization?
- At what level of importance is optimization compared to the other priorities in programming? Why do you think I repeated that question?

Use the Concepts

- Go back through each program in this book and try to make optimizations according to the procedures outlined in this chapter
- Pick a program from the previous exercise and try to calculate the performance impact on your code under specific inputs.²

Going Further

- Find an open-source program that you find particularly fast. Contact one of the developers and ask about what kinds of optimizations they performed to improve the speed.
- Find an open-source program that you find particularly slow, and try to imagine the reasons for the slowness. Then, download the code and try to profile it using `gprof` or similar tool. Find where the code is spending the majority of the time and try to optimize it. Was the reason for the slowness different than you imagined?
- Has the compiler eliminated the need for local optimizations? Why or why not?
- What kind of problems might a compiler run in to if it tried to optimize code across function call boundaries?

2. Since these programs are usually short enough not to have noticeable performance problems, looping through the program thousands of times will exaggerate the time it takes to run enough to make calculations.

